

Markov Process

A state s_t is Markov $\Leftrightarrow p(s_t | s_{t-1}, \dots, s_1) = p(s_t | s_{t-1})$

- One the state is known, the history may be thrown away
- i.e., The state is a sufficient statistic of the future

Markov Process is a memoryless random process

Tuple $\langle S, P, \gamma \rangle$: S is a (finite) set of states

P is state transition probabilities between states
 $\Rightarrow P_{s,s'} = P(s_{t+1} = s' | s_t = s)$

$$P = \begin{bmatrix} p(s_1|s_1) & p(s_2|s_1) & \dots & p(s_N|s_1) \\ p(s_1|s_2) & p(s_2|s_2) & \dots & p(s_N|s_2) \\ \vdots & \vdots & \ddots & \vdots \\ p(s_1|s_N) & \dots & \dots & p(s_N|s_N) \end{bmatrix}$$

Episode: sequence of actions and reward

Episode task: Interaction breaks naturally into episodes

(e.g. play of a game)

Total Reward: $G_t = r_t + \gamma r_{t+1} + \dots + \gamma r_T$

Continuing tasks: Interaction does not have natural episodes, but just goes on and on
= just the real life

In continuing tasks, we often use simple total discounted reward:

$$G_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots = \sum_{k=0}^{\infty} \gamma^k r_{t+k}$$

(discounting)

Markov Reward Process

Markov random process with values

tuple $\langle S, P, r, \gamma \rangle$

S, P : Same as Markov Chain

r : reward function: $r_t = R(s_t, s_{t-1}) = E[r_t | s_t, s_{t-1}]$

γ : discount factor

Horizon

Number of Maximum steps in each episode $\rightarrow$ finite

Return $\rightarrow$ infinite

$G_t = r_t + \gamma r_{t+1} + \dots + \sum_{k=0}^{\infty} \gamma^k r_{t+k}$ "total discounted reward"

Why Discount Factor γ

Most of the Markov reward and decision processes are discounted. Why?

- Avoid infinite returns in cyclic Markov processes
- It is sometimes possible to use undiscounted Markov reward processes (i.e., $\gamma = 1$), e.g., if all sequences terminate
- Mathematically convenient to discount reward
- Uncertainty about the future may not be fully represented

Value Function: $V(s)$

$$V(s) = E[G_t | s_t = s]$$

Bellman Equation

$$V(s) = E[r_t + \gamma V(s_{t+1}) | s_t = s]$$

$$= E[r_t + \gamma V(s_{t+1}) | s_t = s]$$

$$V(s) = r + \gamma \sum_{s'} p(s' | s) V(s')$$

immediate reward discount sum of future reward

Q: Can we calculate all values in a parallel way?

- The Bellman equation can be expressed concisely using matrices

$$\begin{bmatrix} V(s_1) \\ V(s_2) \\ \vdots \\ V(s_N) \end{bmatrix} = \begin{bmatrix} r(s_1) \\ r(s_2) \\ \vdots \\ r(s_N) \end{bmatrix} + \gamma \begin{bmatrix} p(s_1|s_1) & p(s_2|s_1) & \dots & p(s_N|s_1) \\ p(s_1|s_2) & p(s_2|s_2) & \dots & p(s_N|s_2) \\ \vdots & \vdots & \ddots & \vdots \\ p(s_1|s_N) & \dots & \dots & p(s_N|s_N) \end{bmatrix} \begin{bmatrix} V(s_1) \\ V(s_2) \\ \vdots \\ V(s_N) \end{bmatrix}$$

- The Bellman equation can be expressed concisely using matrices

$$V = r + \gamma \cdot P \cdot V$$

$$V = (I - \gamma P)^{-1} r \rightarrow \text{can be computed in } O(N^3)$$

$N = \# \text{ states}$

Markov Decision Process

- S is a (finite) set of states
- A is a (finite) set of actions
- P is a state transition probability matrix, $P_{s,s'}^a = p(s_{t+1} = s' | s_t = s, a_t = a)$
- r is a reward function: $r_t = R(s_t, s_{t-1}, a_t) = E[r_t | s_t = s, a_t = a]$
- γ is a discount factor, $\gamma \in [0, 1]$

MARP with decisions

Tuple $\langle S, A, P, r, \gamma \rangle$

Markov Decision Process: Alternative Definition

Definition: Markov Decision Process
A Markov Decision Process is a tuple $\langle S, A, P, r, \gamma \rangle$:
• S is a (finite) set of states
• A is a (finite) set of actions
• P is a joint probability of a reward r and next state s' , given a state s and action a : $P(r_t, s_{t+1} = s' | s_t = s, a_t = a) = E[r_t | s_t = s, a_t = a]$
• γ is a discount factor, $\gamma \in [0, 1]$

- P defines the dynamics of the problem
- There are equivalent formulations: no additional assumptions:
 $p(s'|s,a) = \sum_i p(s', r_i | s, a)$ and $r(s,a) = \sum_i r_i p(s', r_i | s, a)$

Policy: $\pi(a|s) = P(a = a | s_t = s)$

MDP: $\langle S, A, P, r, \gamma \rangle$

"MP" "MARP": $\begin{cases} P_a = \sum_{s'} P_{s,s'}^a \pi(a|s) p(s'|s,a) \\ P_a(s) = \sum_{s'} P_{s,s'}^a \pi(a|s) r(s',a) \end{cases}$

Value function for MDP

① State-value function

$$V_\pi(s) = E_\pi[G_t | s_t = s], \text{ where } G_t = r_t + \gamma V_\pi(s_{t+1}) + \gamma^2 V_\pi(s_{t+2}) + \dots$$

"expected return following π from state s "

② Action-value function

$$Q_\pi(s,a) = E_\pi[G_t | s_t = s, a_t = a] \text{ "from state } s, \text{ with action } a, \text{ following } \pi"$$

$$\Rightarrow (1) V_\pi(s) = \sum_{a \in A} \pi(a|s) Q_\pi(s,a)$$

$$(2) Q_\pi(s,a) = r(s,a) + \gamma \sum_{s'} p(s'|s,a) V_\pi(s')$$

Bellman Expectation Equation in MDP

① State-value function $\rightarrow$ immediate reward (current state)
discounted reward of successor state
 $\Rightarrow V_\pi(s) = E_\pi[r_t + \gamma V_\pi(s_{t+1}) | s_t = s]$
" $s_t \xrightarrow{\pi} s_{t+1}$ "
 $E_\pi[r_t + \gamma V_\pi(s_{t+1}) | s_t = s]$
② $Q_\pi(s,a) = E_\pi[r_t + \gamma Q_\pi(s_{t+1}, a_{t+1}) | s_t = s, a_t = a]$
" $s_t \xrightarrow{\pi} s_{t+1} \xrightarrow{\pi} s_{t+2} \dots$ "
 $r_t + \gamma Q_\pi(s_{t+1}, a_{t+1}) + \gamma^2 Q_\pi(s_{t+2}, a_{t+2}) + \dots$

$$\Rightarrow V_\pi(s) = \sum_{a \in A} \pi(a|s) (r(s,a) + \gamma \sum_{s'} p(s'|s,a) V_\pi(s'))$$

$$\begin{matrix} & s & \\ a_1 \swarrow & & \searrow a_2 \\ s_1 & & s_2 \\ (\text{prob } p_1) & & (\text{prob } p_2) \end{matrix} \quad \begin{matrix} \uparrow \\ r(s,a) \\ \uparrow \\ \gamma p_1 V_\pi(s_1) + \gamma p_2 V_\pi(s_2) \end{matrix}$$

$$\Rightarrow Q_\pi(s,a) = r(s,a) + \gamma \sum_{s'} p(s'|s,a) \sum_{a' \in A} \pi(a'|s') \cdot Q_\pi(s',a')$$

Theorem

Given an MDP, $\langle S, A, P, r, \gamma \rangle$, for any policy π , their value functions obey the following expectation equations:

$$V_\pi(s) = E_\pi[r_t + \gamma V_\pi(s_{t+1}) | s_t = s]$$

$$Q_\pi(s,a) = E_\pi[r_t + \gamma Q_\pi(s_{t+1}, a_{t+1}) | s_t = s, a_t = a]$$

More detailed:

$$V_\pi(s) = \sum_{a \in A} \pi(a|s) \left(r(s,a) + \gamma \sum_{s'} p(s'|s,a) V_\pi(s') \right)$$

$$Q_\pi(s,a) = r(s,a) + \gamma \sum_{s'} p(s'|s,a) \sum_{a' \in A} \pi(a'|s') Q_\pi(s',a')$$

Matrix Expression of Bellman Equation

For a given policy π , the Bellman expected equation can also be expressed concisely using the induced MRP:

$$V_\pi = r_\pi + \gamma P_\pi V_\pi$$

with the direct solution

$$V_\pi = (I - \gamma P_\pi)^{-1} r_\pi$$

Optimal Value Function

① state-value: $V_*(s) = \max_a V_*(s,a)$
② action-value: $Q_*(s,a) = \max_a Q_*(s,a)$

An MDP is "solved" when we know them

* $V_*(s) = \max_a Q_*(s,a)$

Optimal Policy

Def: partial ordering over policies

$$\pi \succeq \pi' \text{ if } V_\pi(s) \geq V_{\pi'}(s) \forall s$$

Corollaries

For any Markov Decision Process

- There exists at least an optimal policy π^* , that is better than or equal to all other policies, $\pi^* \succeq \pi \forall \pi$
- In other words, there can be multiple optimal policies
- All optimal policies achieve the optimal value function $V_*(s) = V_{\pi^*}(s)$
- All optimal policies achieve the optimal action-value function, $Q_*(s,a) = Q_{\pi^*}(s,a)$

Finding an optimal policy

if we can find $Q^*(s,a)$

then: $\pi^*(a|s) = \begin{cases} 1, & a = \arg\max_a Q_*(s,a) \\ 0, & \text{otherwise} \end{cases}$

"Keep choosing a with max $Q_*(s,a)$ "

if we know $V_*(s)$

$$s \begin{matrix} \swarrow s_1 \\ \searrow s_2 \\ \vdots \end{matrix} \rightarrow \text{need to look one step ahead} \rightarrow \text{where "a" lead to } s^*?$$

Policy Search

① enumeration: $\# |A|^{|S|}$

② policy iteration / value iteration

Policy Evaluation

Given π , estimate V_π, Q_π

"prediction"

- Estimating V_π or Q_π is sometimes called control, because these can be used for policy optimization
- What is the optimal way of behaving? What is the optimal value function?

	State Values	Action Values
Prediction	V_π	Q_π
Control	V_π	Q_π

$$\begin{cases} V_\pi(s) = \max_a [r(s,a) + \gamma \sum_{s'} p(s'|s,a) V_\pi(s')] \\ Q_\pi(s,a) = \max_a [r(s,a) + \gamma \sum_{s'} p(s'|s,a) \max_{a'} Q_\pi(s',a')] \end{cases}$$

Policy Evaluation

$$V_\pi(s) = E_\pi[r_t + \gamma V_\pi(s_{t+1})]$$

Using Dynamic Programming

Algorithm

- Initialize V_a
 - $\forall s, V_{a,t+1}(s) \leftarrow E[r_t + \gamma V_a(s_{t+1}) | s_t = s]$
 - Stopping when $V_{a,t+1} = V_a$ i.e. Convergence.
- In practice: $V_{a,t+1}(s) = \sum_{a' \in A} \pi(a'|s) (r(s,a') + \gamma \sum_{s'} p(s'|s,a') V_a(s'))$

Iterative Policy Evaluation, for estimating $V_\pi = v_\pi$
Input π , the policy to be evaluated
Algorithm parameter: a small threshold $\theta > 0$ determining accuracy of estimation
Initialize $V(s)$ arbitrarily, for $s \in S$, and V (terminal) to 0
Loop:
 $\Delta \leftarrow 0$
Loop for each $s \in S$:
 $v \leftarrow V(s)$
 $V(s) \leftarrow \sum_{a \in A} \pi(a|s) \sum_{s'} p(s'|s,a) (r(s,a) + \gamma V(s'))$
 $\Delta \leftarrow \max_s |\Delta - (V(s) - v)| \leftarrow \text{Update } V_{a,t+1} = V_a$
until $\Delta < \theta$

Policy Improvement

given π , objective: Finding $\pi' \succeq \pi$

Solution: given a deterministic policy

i.e. $\pi(s) = a$

Improve it Greedily

$$\pi^*(s) = \arg \max_a Q_\pi(s,a) \\ = \arg \max_a E[r(s,a) + \gamma V_\pi(s_{t+1}) | s_t = s, a_t = a]$$

$$\pi^*(s) = \arg \max_a Q_\pi(s,a) \Rightarrow Q(s, \pi^*(s)) \geq V_\pi(s)$$

$$V_\pi(s) \leq Q(s, \pi^*(s)) = E[r_t + \gamma V_\pi(s_{t+1}) | s_t = s, a_t = \pi^*(s_{t+1})]$$

$$= E_\pi[r_t + \gamma V_\pi(s_{t+1}) | s_t = s]$$

$$\leq E_\pi[r_t + \gamma Q_\pi(s_{t+1}, \pi^*(s_{t+1})) | s_t = s]$$

$$= E_\pi[r_t + \gamma \pi(s_{t+1}) + \gamma V_\pi(s_{t+2}) | s_t = s]$$

$$\leq E_\pi[r_t + \gamma \pi(s_{t+1}) + \gamma^2 V_\pi(s_{t+2}) | s_t = s]$$

$$= V_\pi(s)$$

$$\begin{aligned} Q_\pi(s,a) &\leq Q_\pi(s, \pi^*(s)) \\ &= E_\pi[r_t + \gamma Q_\pi(s_{t+1}, a_{t+1}) | s_t = s, a_t = \pi^*(s_{t+1})] \\ &\leq E_\pi[r_t + \gamma Q_\pi(s_{t+1}, \pi^*(s_{t+1})) | s_t = s, a_t = \pi^*(s_{t+1})] \\ &= E_\pi[r_t + \gamma E[r_{t+1} + \gamma Q_\pi(s_{t+2}, a_{t+2}) | s_{t+1} = s, a_{t+1} = \pi^*(s_{t+2})] | s_t = s, a_t = \pi^*(s_{t+1})] \\ &\leq E_\pi[r_t + \gamma r_{t+1} + \gamma^2 Q_\pi(s_{t+2}, \pi^*(s_{t+2})) | s_t = s, a_t = \pi^*(s_{t+1})] \\ &\leq E_\pi[r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots | s_t = s, a_t = \pi^*(s_{t+1})] \\ &= Q_\pi(s, a_t = \pi^*(s)) \end{aligned}$$

So far, summary: Given a policy π

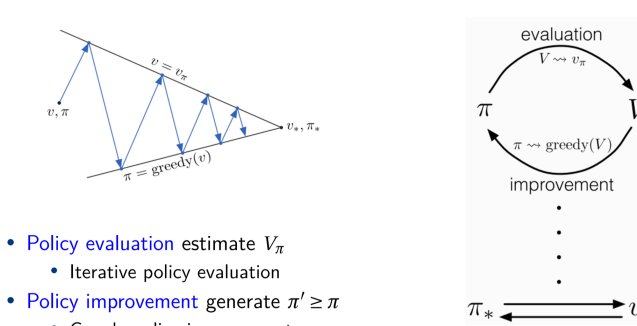
- Evaluate the policy π

$$V_\pi(s) = E[r_t + \gamma r_{t+1} + \dots | s_t = s]$$

- Improve the policy by acting greedily with respect to V_π
 $\pi' = \text{greedy}(V_\pi)$

Policy iteration

$\pi \rightarrow \text{Evaluating } V_\pi \rightarrow \text{improving } \pi \rightarrow E \rightarrow V_\pi \rightarrow \pi \rightarrow E \rightarrow \dots$



- Policy evaluation estimate V_π
 - Iterative policy evaluation
- Policy improvement generate $\pi' \succeq \pi$
 - Greedy policy improvement

Policy Iteration (using iterative policy evaluation) for estimating $\pi^* = \pi^*$
1. Initialization
 $V(s) \in \mathbb{R}$ and $\pi(s) \in A(s)$ arbitrarily for all $s \in S$; V (terminal) = 0
2. Policy Evaluation
Loop:
 $\Delta \leftarrow 0$
Loop for each $s \in S$:
 $v \leftarrow V(s)$
 $V(s) \leftarrow \sum_{a \in A} \pi(a|s) \sum_{s'} p(s'|s,a) (r(s,a) + \gamma V(s'))$
 $\Delta \leftarrow \max_s |\Delta - (V(s) - v)|$
until $\Delta < \theta$ (a small positive number determining the accuracy of estimation)
3. Policy Improvement
policy-stable = true
For each $s \in S$:
old-action = $\pi(s)$
 $\pi(s) \leftarrow \arg \max_a \sum_{s'} p(s'|s,a) (r(s,a) + \gamma V(s'))$
If old-action $\neq \pi(s)$, then policy-stable = false
If policy-stable, then stop and return V and π ; else $\pi \leftarrow \pi'$, else go to 2

Actually, we need not to let V_a converge in practice e.g. let $k=3, n>k$ stop

Why not update policy every iteration?

Value Iteration

$$V_*(s) = \max_a \sum_{s'} p(s'|s,a) [r(s,a) + \gamma V_*(s')]$$

Value Iteration directly takes the Bellman optimality equation, and turns that into an update

Policy Iteration
1. Evaluation $V_\pi(s)$ (LOOP inside) // policy evaluation
2. $\pi'(a|s) = \begin{cases} 1, & \text{if } a_t = \arg \max_a Q_\pi(s,a) \\ 0, & \text{otherwise} \end{cases}$
3. Set $\pi \leftarrow \pi'$ (LOOP) GO TO STEP1
Value Iteration
1. Set $Q(s,a) = r(s,a) + \gamma E_{\pi'}[r(s,a) | s_t = s, a_t = a]$
2. Set $V(s) = \max_a Q(s,a)$ (LOOP) GO TO STEP1

- Algorithms are based on state-value function $V_\pi(s)$ or $V_*(s)$
 - Complexity $O(mn^2)$ per iteration, for m actions and n states
- Could also apply to action-value function $Q_\pi(s,a)$ or $Q_*(s,a)$
 - Complexity $O(m^2n^2)$ per iteration
- To find an optimal policy is polynomial in the number of states
- But, the number of states is often astronomical, e.g., often growing exponentially with the number of state variables ("curse of dimensionality")
- In practice, classical DP can be applied to problems with a few million states

Classical DP

$$V_{\pi^*}(s) = \max_a [r(s,a) + \gamma \sum_{s'} p(s'|s,a) V_{\pi^*}(s')] \text{ For All } s$$

if $\# \text{ states}$ is large, then computation/storage $\uparrow$

Synchronous DP

- idea: ① Sample a state s at random
② Update $V_\pi(s)$ ONLY (instead of all states)

Still Lots of Computation But better than Classical DP

- Synchronous value iteration stores two copies of the value function
- For all s in S

$$V_{\text{new}}(s) = \max_a \left(r(s,a) + \gamma \sum_{s'} p(s'|s,a) V_{\text{old}}(s') \right) \\ V_{\text{old}}(s) \leftarrow V_{\text{new}}(s)$$

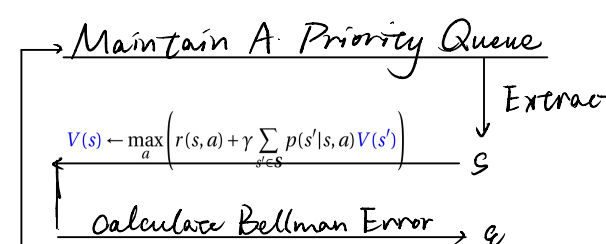
- In-place value iteration only stores one copy of the value function
- For all s in S

$$V(s) \leftarrow \max_a \left(r(s,a) + \gamma \sum_{s'} p(s'|s,a) V(s') \right)$$

- Use the magnitude of Bellman error to guide state selection, e.g.,

$$\max_a \left(r(s,a) + \gamma \sum_{s'} p(s'|s,a) V(s') \right) - V(s)$$

- Backup the state with the largest remaining Bellman error
- Update Bellman of affected states after each backup
- Requires knowledge of reverse dynamics (predecessor states)
- Can be implemented efficiently by maintaining a priority queue



- Idea: Focus on states that are relevant to the agent (we need an agent to interact with the world, to guide the priority over backup updates)
- Use agents experience to guide the selection of states
- After each time-step s_t, a_t, r_{t+1}
- Backup the state s

$$V(s_t) \leftarrow \max_a \left(r(s_t, a_t) + \gamma \sum_{s'} p(s'|s_t, a_t) V(s') \right)$$

Contraction Mapping Theorem